

Ros2 Day2 hw2 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. 실행결과

1. 프로젝트 개요

이 프로젝트는 Day1에서 키보드로 조작했던 터틀심 그림 그리기를 Qt로 만든 창에서 버튼을 눌러 조작하도록 옮긴 것이다. 파이썬으로 만들었던 것을 C++로 다시 작성했고, 터미널 대신 화면의 버튼을 누르면 거북이가 도형을 그린다.

사용하는 통신은 Day1과 같다. 거북이를 움직이는 속도 토픽과 펜의 색상과 굵기를 바꾸는 서비스를 쓰고, 터틀심 노드가 화면 표시를 담당한다.

창에는 W, A, S, D 네 개의 버튼과 라벨 하나를 두었다. 각 버튼을 누르면 삼각형, 원, 사각형, 별이 그려지고, 도형마다 펜의 색상과 굵기를 다르게 설정해두었다. 라벨에는 현재 발행

되고 있는 속도 값이 실시간으로 표시된다.

화면 쪽은 버튼이 눌렸다는 것만 알려주고 실제 발행은 통신 쪽에서 하며, 반대로 속도 값이 바뀌면 통신 쪽이 신호를 보내 화면의 라벨을 갱신한다. 두 부분이 서로 다른 스레드에서 돌기 때문에 값을 직접 건드리지 않고 신호를 주고받는 방식으로 연결하였다.

2. 코드 구조

```
class QNode : public QThread
{
    Q_OBJECT
public:
    ...
private:
    std::shared_ptr<rclcpp::Node> node;
    rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr cmd_vel_pub_;
    rclcpp::Client<turtlesim::srv::SetPen>::SharedPtr set_pen_client_;
Q_SIGNALS:
    void rosShutDown();
    void cmdVelUpdated(double linear, double angular);
};
```

ROS 통신을 담당하는 클래스. 스레드를 상속받고 노드를 멤버로 들고 있으며, 속도를 발행할 퍼블리셔와 펜 설정을 요청할 클라이언트를 가진다. 아래쪽 두 개는 화면 쪽으로 보낼 신호다.

```
QNode::QNode()
{
    int argc = 0;
    char** argv = NULL;
    rclcpp::init(argc, argv);
    node = rclcpp::Node::make_shared("juwon_turtle_gui");
    cmd_vel_pub_ =
node->create_publisher<geometry_msgs::msg::Twist>("/turtle1/cmd_vel", 10);
    set_pen_client_ = node->create_client<turtlesim::srv::SetPen>("/turtle1/set_pen");
    this->start();
}
```

```
}
```

노드를 만들고 퍼블리셔와 클라이언트를 등록한 뒤 마지막에 스레드를 시작하는 부분. 등록보다 시작이 먼저 오면 아직 만들어지지 않은 것을 쓰게 되므로 시작은 반드시 맨 아래에 둔다.

```
void QNode::publish_cmd_vel(double linear, double angular)
```

속도를 발행하는 부분. 모든 움직임이 이 함수를 거쳐 가도록 만들어서 발행 코드가 중복되지 않게 했고, 발행할 때마다 화면 쪽으로 신호를 보내 라벨이 갱신되도록 했다.

```
void QNode::straight() //직진
```

```
{
```

```
    publish_cmd_vel(2.0, 0.0);
```

```
    QThread::msleep(1000);
```

```
}
```

```
void QNode::rotate120() //120도 회전
```

```
{
```

```
    publish_cmd_vel(0.0, 2.094);
```

```
    QThread::msleep(1000);
```

```
}
```

직진과 회전을 각각 함수로 빼둔 부분. 값만 다르고 구조는 같으며, 도형마다 필요한 각도에 맞춰 회전 함수를 따로 만들었다.

```
void QNode::set_pen(int r, int g, int b, int width)
```

```
{
```

```
    auto request = std::make_shared<turtlesim::srv::SetPen::Request>();
```

```
    request->r = r;
```

```
    ...
```

```
    set_pen_client_->async_send_request(request);
```

```
    QThread::msleep(100); // 적용될 시간
```

```
}
```

펜의 색상과 굵기를 바꾸는 요청을 만들어 보내는 부분. 요청이 실제로 적용되기 전에 거북이가 움직여버리면 이전 색으로 그려지기 때문에 잠깐 기다려준다.

```
void QNode::triangle()
```

```
{
```

```

set_pen(255, 0, 0, 6);
for (int i = 0; i < 3; i++)
{
    straight();
    stop();
    rotate120();
    stop();
}
stop();
}

```

펜을 빨간색, 굵기 6으로 바꾸고 세 번 반복하며 직진과 회전을 번갈아 실행하는 부분. 사각형과 별도 각도와 반복 횟수만 다를 뿐 같은 구조다.

```

void QNode::circle()
{
    set_pen(0, 0, 255, 16);
    auto msg = geometry_msgs::msg::Twist();
    msg.linear.x = 10.0;
    msg.angular.z = 7.0;
    cmd_vel_pub_->publish(msg);
    ...
}

```

원은 직진과 회전을 동시에 주어야 하므로 반복 없이 한 번만 발행한다.

```

MainWindow::MainWindow(QWidget* parent) : QMainWindow(parent), ui(new
Ui::MainWindowDesign)
{
    ui->setupUi(this);
    qnode = new QNode();

    connect(ui->btn_w, &QPushButton::clicked, this, [this]() { qnode->triangle(); });
    connect(ui->btn_s, &QPushButton::clicked, this, [this]() { qnode->rectangle(); });
    connect(ui->btn_a, &QPushButton::clicked, this, [this]() { qnode->circle(); });
    connect(ui->btn_d, &QPushButton::clicked, this, [this]() { qnode->star(); });
    connect(qnode, &QNode::cmdVelUpdated, this,
&MainWindow::updateCmdVelLabel);
}

```

```

...
}

```

화면을 만들고 각 버튼을 도형 함수에 연결하는 부분. 마지막 줄은 반대 방향으로, 통신 쪽에서 온 신호를 라벨 갱신 함수에 연결한 것이다. 통신 쪽 객체를 먼저 만들어두어야 연결이 가능하므로 순서가 중요하다.

```

void MainWindow::updateCmdVelLabel(double linear, double angular)
{
    ui->label_cmd_vel->setText(QString("linear.x:%1/ angular.z:%2").arg(linear).arg(angular));
    QApplication::processEvents();
}

```

신호를 받아 라벨의 글자를 바꾸는 부분. 도형을 그리는 동안 화면이 멈춰 있어서 라벨이 갱신되지 않는 문제가 있었는데, 마지막 줄을 넣어 갱신을 강제로 처리하게 하여 해결했다.

3. 파일 구조

```

juwon_turtle_gui
├── include
│   ├── juwon_turtle_gui
│   │   ├── main_window.hpp
│   │   └── qnode.hpp
├── src
│   ├── main.cpp
│   ├── main_window.cpp
│   └── qnode.cpp
├── ui
│   └── mainwindow.ui
├── resource
├── CMakeLists.txt
└── package.xml

```

CMakeLists.txt에서는 Qt 관련 패키지와 함께 속도 메시지, 펜 서비스가 들어 있는 패키지를 의존성으로 걸어준다. 처음에는 이 두 개가 빠져 있어서 빌드가 되지 않았다.

4. 실행 결과

첨부 영상 참고